

DiRe – JAX: A JAX based Dimensionality Reduction Algorithm for Large-scale Data

Alexander Kolpakov¹ and Igor Rivin²

¹University of Austin, Austin TX, USA; akolpakov@uaustin.org

²Temple University, Philadelphia PA, USA; rivin@temple.edu

Abstract

Summary: DiRe – JAX is a new dimensionality reduction toolkit designed to address some of the challenges faced by traditional methods like UMAP and tSNE such as loss of global structure and computational efficiency. Built on the JAX framework, DiRe leverages modern hardware acceleration to provide an efficient, scalable, and interpretable solution for visualizing complex data structures, and for quantitative analysis of lower-dimensional embeddings. The toolkit shows considerable promise in preserving both local and global structures within the data as compared to state-of-the-art UMAP and tSNE implementations. This makes it suitable for a wide range of applications in machine learning, bio-informatics, and data science.

1 Statement of Need

Traditional dimensionality reduction techniques such as UMAP and tSNE are widely used for visualizing high-dimensional data in lower-dimensional spaces, usually 2D and sometimes 3D. Other uses include dimensionality reduction to other, possibly higher and thus non-visual dimensions, for the subsequent use of classifiers such as SVMs.

However, these methods often struggle with scalability, interpretability, and preservation of global data structures. UMAP, while fast and scalable, may overemphasize local structures at the expense of global data relationships [CP23]. And tSNE, while known for producing high-quality visualizations, may be computationally expensive and sensitive to hyperparameter tuning [KB19].

DiRe-JAX addresses these challenges by offering a scalable solution that balances the preservation of both local and global structures. Leveraging the JAX framework allows DiRe-JAX to efficiently handle large datasets by utilizing GPU / TPU acceleration, making it significantly faster than CPU-based implementations without compromising on the quality of the embeddings.

Also, DiRe-JAX includes a wealth of metrics that can be used to analyze the quality of embeddings, and for hyperparameter tuning. Given the runtime efficiency of DiRe-JAX, such optimization tasks become achievable in relatively simple and low-cost environments such as Google Colab.

This makes DiRe-JAX an essential toolkit for researchers and practitioners working with complex, high-dimensional data.

2 Main methods

The main class of DiRe-JAX is `DiRe`. Let $X \subset \mathbb{R}^n$ be the input data realized as a numpy array. Then, `DiRe` performs the following main steps:

1. **Capturing dataset topology:** Create the kNN graph of X , say Γ , for a given number of neighbors k (`= n_neighbors`) by calling `make_knn_adjacency`. This step uses a JAX kernel specifically developed by the authors in order to perform the computation in the CPU, GPU or TPU settings. Other libraries like FAISS [Dou+24] may also be used, although they do not have the same level of hardware universality.
2. **Initial dimension reduction:** Produce $Y \subset \mathbb{R}^d$, the initial embedding of X , with $d \ll n$ (usually $d = 2$ or 3) given by `dimension` by using one of the available embedding methods: 'random' (random projections from the Johnson–Lindenstrauss Lemma), 'spectral' (using the kNN graph Γ to construct the weighted Laplacian optionally applying a similarity kernel), or 'pca' (classical or kernel-based).
3. **Layout optimization:** Calling `do_layout` to adjust the lower-dimensional embedding Y to conform to the similarity structure of the higher-dimensional data X . This is done by using a force layout where the role of “forces” is played by probability kernels (the distribution can be adjusted via parameters `min_dist` and `spread`).

The initial embedding is stored in `self.init_embedding`, while the optimized layout is stored in `self.layout`. Both can be accessed after the main method `fit_transform` is called, for a more detailed study and comparison.

The initial embedding can be one of the following three choices, each of which we discuss in more detail.

2.1 Random Projection embedding

This embedding is based on the following famous lemma (in fact, a rather useful theorem) of Johnson and Lindenstrauss [JL84].

Johnson–Lindenstrauss Lemma (Probabilistic Form)

Given $0 < \epsilon < 1$ and an integer n , let X be a set of n points in $\mathbb{R}^d$. For a random linear map $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$ where $k = O\left(\frac{\log n}{\epsilon^2}\right)$, with high probability, for all $u, v \in X$,

$$(1 - \epsilon)\|u - v\|^2 \leq \|f(u) - f(v)\|^2 \leq (1 + \epsilon)\|u - v\|^2.$$

The value

$$\text{dist}(f) = \frac{\|f(u) - f(v)\|}{\|u - v\|}$$

is called the *distortion* of f , and is expected to be close to 1.0 for a good quality embedding.

The Johnson–Lindenstrauss Lemma says that a random projection from $\mathbb{R}^d$ to $\mathbb{R}^k$ has, in general, its distortion close to 1. Notice, however, that generally the distortion does not have to be small

when k is in the human-intelligible visualization range ($k = 1, 2, 3$). On the other hand, this method is simple and computationally inexpensive.

One problem with projections, though, is that the image of the map $p : \mathbb{R}^d \rightarrow \mathbb{R}^k$ is likely to be very cluttered if $k \ll d$. Indeed, the total variance of $Y = p(X) \in \mathbb{R}^{n \times k}$ may be much lower than the total variance of $X \in \mathbb{R}^{n \times d}$ unless we project onto the first k largest variance components, so that the variance matrices $Y^t Y$ and $X^t X$ have close spectral norms.

2.2 Principal Component Analysis embedding

The discussion above indicates that the best way of preserving the internal structure of the dataset is projecting it in a way that preserves as much of its variance as possible. Such method is classically known as Principal Component Analysis.

Assume that X is normalized, so that the columns of X have each zero mean. Let its covariation matrix be $\text{Cov}(X) = \frac{1}{n-1} X^t X$. The best strategy here is computing the singular decomposition of X , so that

$$X = U \Sigma W^t,$$

where Σ has $k \ll d$ dominant singular values $\Sigma_k = \langle \sigma_1, \dots, \sigma_k \rangle$ with

$$\sum_{i=1}^k \sigma_i^2 = (1 - \varepsilon)^2 \|X\|_F^2.$$

Let Σ_k be rank k truncation of Σ preserving the dominant singular values above, and let U_k and W_k be the respective truncation of U and W . Then the rank k approximation of X is

$$\hat{X} = U_k \Sigma_k W_k^t$$

and the PCA embedding of X is

$$X_k = X W_k.$$

If we believe that most of the topological features of X are expressed in dimensions up to k , while the remaining dimensions are negligible, we then contend that the persistence diagrams $D(X)$ of X in the first k dimensions may be replaced by those of $\hat{X}$. The actual k -dimensional image of $\hat{X}$ is $\hat{X} W_k$. Thus, in the first k dimensions we put $D(X) := D(\hat{X} W_k)$. We naturally have this equality if X belongs to an affine k -dimensional subspace of $\mathbb{R}^d$.

Then, according to [CSO14], for the persistence diagrams we have that the bottleneck distance between the persistence diagrams of X and X_k or, equivalently, X_k satisfies

$$d_b(D(X), D(X_k)) = d_b(D(\hat{X} W_k), D(X W_k)) \leq \|\hat{X} W_k - X W_k\|_F = \|\hat{X} - X\|_F = \varepsilon \|X\|_F.$$

In this sense, the persistence homology of X is preserved, up to some extent, under the PCA embedding (cf. Section 3 for basic facts on persistence homology and associated measures).

2.3 Spectral Laplacian embedding

Yet another popular embedding method which may or may not preserve the internal structure of the data. It is often geared towards manifold learning of the local data structure in order to deal with non-linearity, even though unwrapping non-linear data in lower dimensions may or may not be beneficial for its subsequent analysis. The same applies to diffusion maps. Since the kNN -graph Γ of X is already available, the Laplacian embedding can be computed with standard methods.

2.4 Force-directed layout

After the initial embedding, DiRe-JAX applies an iterative force-directed layout in order to correlate the local structure of the embedding Y with the local structure of the high-dimensional dataset X . The forces of attraction and repulsion are given by simple distributions modeled after the t -distributions of UMAP and tSNE.

Namely, a simple density function

$$\varphi(x) = \frac{1}{1 + a \|x\|^{2b}}$$

is fit to `min_dist` = δ and `spread` = σ parameters, so that

$$\varphi(x) \approx 1.0$$

within $\|x\| < \delta$, and then decays exponentially as

$$\varphi(x) \approx \exp(-(\|x\| - \delta)/\sigma)$$

outside of the δ -ball.

The attraction forces are defined between the point in Y that are projections of kNN -neighbors in Γ , of magnitude

$$\varphi\left(\frac{1}{\|y_i - y_j\|}\right), \text{ for } y_i, y_j \in Y \text{ such that } (y_i, y_j) \in \Gamma.$$

All other pair of points experience a repulsion force of magnitude

$$\varphi(\|y_i - y_j\|) \text{ for } y_i, y_j \in Y \text{ such that } (y_i, y_j) \notin \Gamma.$$

The layout stops moving points after a given number of steps that is preset as a parameter. This has an effect on the persistence homology of Y as compared to that of X . In our numerical experiments we shall see, however, that the difference manifests itself largely as “time dilation” with respect to the filtration level, meaning that such metrics as the Dynamic Time Warp distance or Time Warp Edit Distance would still be close enough.

3 Quantitative measures

We use several quantitative measures to assess the quality of DiRe-JAX embeddings, and to compare our algorithm to other embedding techniques available, such as tSNE (in its cuML implementation), as well as UMAP (in both its original and cuML implementations).

3.1 Measuring the global structure

We use several types of measures for persistence homology. One type is related to *persistence diagrams*, and the other is related to the *Betti curves* derived from them. Although basically equivalent, these two types of persistence homology representation happen to be useful in different ways.

3.1.1 Persistence diagrams

Persistence diagrams are classical representations of the “birth–death” pairs in persistence homology. The reader may find more information in [ZC05; Zom05], while we give a brief description below.

The *Victoris–Rips complex* at scale $t \geq 0$, denoted $\text{VR}_t(X)$, is the simplicial complex defined as

$$\text{VR}_t(X) = \{\sigma \subseteq X \mid \|x_i - x_j\| \leq t \text{ for all } x_i, x_j \in \sigma\},$$

which means a simplex σ is contained in $\text{VR}_t(X)$ if all its vertices are pairwise within distance t .

Consider a non-decreasing sequence of scales $\{t_i\}_{i \in I}$, where I is an index set (often $I = \mathbb{R}_{\geq 0}$), generating a *filtration* of Rips complexes:

$$\text{VR}_{t_0}(X) \subseteq \text{VR}_{t_1}(X) \subseteq \text{VR}_{t_2}(X) \subseteq \cdots.$$

For each $k \geq 0$ and scale t_i , compute the k -th homology group $H_k(\text{VR}_{t_i}(X); \mathbb{F})$ with coefficients in a field $\mathbb{F}$ (commonly $\mathbb{F} = \mathbb{Z}/2\mathbb{Z}$, or another finite field). The inclusion maps induce homomorphisms between homology groups:

$$f_{s,t}^k : H_k(\text{VR}_s(X); \mathbb{F}) \rightarrow H_k(\text{VR}_t(X); \mathbb{F}), \quad \text{for } s \leq t.$$

A persistent k -dimensional homology class α is an element of $H_k(\text{VR}_s(X); \mathbb{F})$ that persists across multiple scales. The *birth time* b_α and *death time* d_α of α are defined as

- b_α : the smallest t where $\alpha \neq 0$ appears in $H_k(\text{VR}_t(X); \mathbb{F})$;
- d_α : the smallest $t > b_\alpha$ where $f_{b_\alpha,t}^k(\alpha) = 0$.

For homology classes that persist indefinitely, we set their death time to infinity: $d_\alpha = \infty$.

The *persistence diagram* D_k is the (multi-)set of points (b_α, d_α) in the extended plane $\mathbb{R}^2 \cup \{\infty\}$:

$$D_k(X) = \{(b_\alpha, d_\alpha) \in (\mathbb{R} \times (\mathbb{R} \cup \{\infty\})) \mid \alpha \in H_k(X) \text{ with birth } b_\alpha \text{ and death } d_\alpha\},$$

where we use $H_k(X)$ to denote the k -dimensional persistence homology group defined above.

Features with longer lifespans $d_\alpha - b_\alpha$ are considered topologically significant. In particular, features with $d_\alpha = \infty$ represent persistent homological features that never disappear within the considered scale range, indicating essential topological structures of the space.

The usual metrics between two diagrams $D = D_k(X)$ and $D' = D_k(Y)$ are the bottleneck and Wasserstein distances, cf. [Per24].

3.1.2 Betti curves

Betti curves mark the progression of the persistence Betti numbers with the filtration level increase. This representation was first studied in detail in [GGB16]. Let X be a point cloud and $\text{Rips}_t(X)$ be its Rips complex of level $t \geq 0$. Then the k -th Betti curve is

$$\beta_k(t) = \text{rank } H_k(\text{VR}_t(X)).$$

The metrics to compare two Betti curves are numerous, and we choose those that eliminate the dependence on the “time-warp” associated with possible reparametrisations of t .

The most common are the Dynamic Time Warp (DTW) distance [SC07; Dev23], Time Warp Edit Distance (TWED) [Mar09; Zum21], and Earth Mover’s Distance (EMD) [Fla+21; POT25].

3.1.3 Global structure preservation

Let $X \in \mathbb{R}^{n \times d}$ be a set of n vectors in $\mathbb{R}^d$ with kNN -graph Γ . Let $Y = f(X) \in \mathbb{R}^{n \times k}$ be its embedding. We shall measure the most common distances between the corresponding persistence diagrams $D_k^H = D_k(X)$ and $D_k^L = D_k(Y)$ of higher-dimensional data and its lower-dimensional embedding, respectively. Namely, we will use the bottleneck $d_b(D_k^H, D_k^L)$ and Wasserstein distances $d_W(D_k^H, D_k^L)$ for $k = 0, 1$, as our embedding will be 2-dimensional in all practical examples.

We use the normalized version of d_b and d_W , where the distance is averaged over the number of points subsampled from X and Y for the feasibility of our computation.

We will also employ the Betti curves $\beta^H(t)$, for the higher-dimensional data, and $\beta^L(t)$, for the lower-dimensional embedding, respectively. The distances computed between $\beta^H(t)$ and $\beta^L(t)$ will be DTW, TWED, and EMD.

Again, we compute their normalized versions: averaged over the number of subsampled points. Also, the curves are first normalized to the unit mass for the EMD computation, and later on the distance is rescaled back. Indeed, we are first interested in how much the mass has to be moved on the relative scale, and the rescaling does not change the essential topology of the mass distribution. If τ is our normalizing factor, we thus compute $\max\{\tau, \tau^{-1}\} \cdot d_{EMD}(\beta^H, \tau \cdot \beta^L)$.

3.2 Measuring the local structure

We use two local measures: embedding stress and neighborhood preservation ratio.

3.2.1 Embedding stress

Let $X \in \mathbb{R}^{n \times d}$ be a set of n vectors in $\mathbb{R}^d$ with kNN -graph Γ . Let $Y = f(X) \in \mathbb{R}^{n \times k}$ be its embedding. For each edge $e = (x, y)$ in Γ we have its local embedding stress measured by

$$\lambda(e) = \left| 1 - \frac{\|x - y\|_2}{\|f(x) - f(y)\|_2} \right|$$

Let $\Lambda = \Lambda(f, \Gamma) = (\lambda(e))_{e \in \text{edges}(\Gamma)}$ be the sequence of local edge stresses. The total embedding stress of Γ under f is then computed as

$$\sigma(f, \Gamma) = \frac{\sqrt{\text{Var}[\Lambda]}}{\mathbb{E}[\Lambda]}.$$

Note that σ is scale-invariant in the sense that if all edge lengths of Γ are being rescaled under f , then $\sigma(f, \Gamma) = 0$, as there is no *relative* change of the internal metric of Γ .

3.2.2 Neighborhood preservation

Given X and its kNN graph $\Gamma_X := \Gamma$, let us consider $Y = f(X)$ and the kNN graph Γ_Y of Y . The neighborhood preservation measure counts how many neighbours of a vertex x_i of a vertex x in Γ_X are mapped into the neighbors y_i of a vertex $y = f(x)$ in Γ_Y . That is, let $N_X(x)$ be the neighbor indices for a vertex x of Γ_X . Let the vertex indices in Y be inherited from X , that is $y_i = f(x_i)$, for x_i a vertex in Γ_X . Let $N_Y(y)$ be the neighbor indices for a vertex y of Γ_Y .

Then the neighborhood preservation sequence of f is given by $N(X, f) = \left(\frac{\#N_X(x)}{\#N_Y(f(x))} \right)_{x \in X}$. The neighborhood preservation index is then the pair

$$\nu(X, f) = \left(\mathbb{E}[N(X, f)], \sqrt{\text{Var}[N(X, f)]} \right).$$

We intentionally compute both the mean and standard deviation in order to see not only how well neighborhoods are preserved, but also how variable the loss of local structure is.

In our numerical experiments, it turns out that the neighborhood structure is very weakly preserved, since any dimension reduction deforms distances (if not greatly then still enough to changes the nearest neighbors completely).

3.3 Measuring context loss

If X is a labeled set of data points, then its *context* is the mutual position (both geometric and topological) of its labeled subsets. We want our dimension reduction method to produce a lower-dimensional embedding Y that preserves the context, and thus presents a human-readable picture that is yet informative of the high-dimensional relationships in X .

3.3.1 Linear SVM classifier accuracy

If X is labeled, we measure how well X is classified by a linear Support Vector Machine. Then we do the same for Y . The main idea is that an embedding $Y = f(X)$ is “good” relative to the context of the data points whenever two clusters A and B separated in X by a hyperplane can also be separated in Y and, conversely, a non-linearity of X is preserved in Y . The latter means that if two subsets A and B of X cannot be separated by an SVM hyperplane, then $f(A)$ and $f(B)$ should not be separable either.

Thus, we expect the accuracy α_X of an SVM classifier trained and tested on X be close to that of one trained and tested on Y , say α_Y . Namely, the SVM context loss is defined by

$$\kappa_{SVM} = \log \min \left\{ \frac{\alpha_X}{\alpha_Y}, \frac{\alpha_Y}{\alpha_X} \right\}$$

A lower, or even higher, accuracy of the latter as compared to the former would mean that some global context of the data points is lost.

3.3.2 kNN classifier accuracy

We also compare the accuracy of a kNN classifier trained and tested on X to that of one trained and tested on Y . Here, the classifier is essentially non-linear, as opposed to the SVM case above. Thus, we want to see if the embedding worsens or improves the accuracy as opposed to its mere preservation.

Let α_X be the accuracy of a kNN classifier trained and tested on X , and let α_Y be the accuracy of a kNN classifier trained and tested on Y . The kNN context loss is then defined as

$$\kappa_{kNN} = \log \frac{\alpha_Y}{\alpha_X}.$$

4 General workflow

DiRe-JAX follows a streamlined workflow that is designed for ease of use and integration into existing data analysis pipelines.

1. **Data Preprocessing:** At the moment DiRe-JAX does not offer any additional data pre-processing tools. Whether the data has to be normalized, standardized, or processed via some transformation for variance reduction (e.g. log- or sinh⁻¹-transform) is the ultimate decision of its user. However, most preprocessing techniques are available via `scikit-learn` or similar packages.
2. **Data embedding:** Performed according to the steps described in the previous section (cf. Methods).
3. **Integration and Export:** DiRe-JAX is designed to be easily integrated into larger machine learning workflows. It can be instantiated and then process the input data via its

`fit_transform` method. It keeps the initial embedding as its `init_embedding` attribute, while the optimized layout is kept separately as `layout`. Also, the layout can be visualized by calling `visualize`.

4. **Embedding metrics:** The final embedding becomes more explainable and quantifiable due to a number of metrics described above (cf. Quantitative Measures). This allows to understand the lower-dimensional embedding goodness, as well as perform hyperparameter optimization.

5 Benchmarks

A collection of benchmarks is available in our GitHub repository [KR25], some of which are given below. We shall concentrate on the main differences between DiRe-JAX, tSNE, and UMAP. Both tSNE and UMAP were implemented by Rapids.ai [Dev25a], and these are the GPU-based implementations that we use for benchmarking. We also use the original UMAP implementation by McInnes et al. [Dev25b].

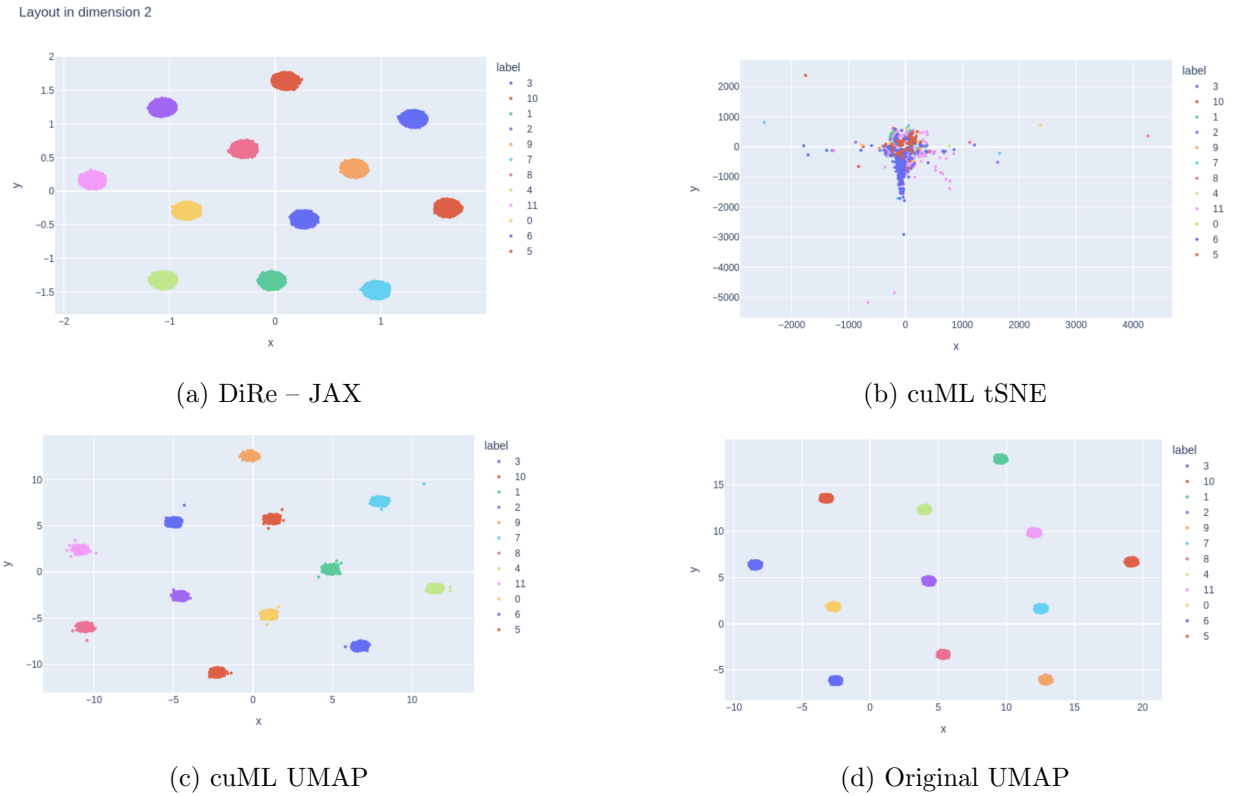


Figure 1: "Blobs" dataset embeddings

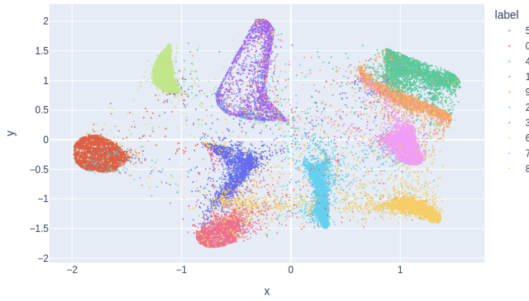
5.1 Dataset: Blobs

The 2D embeddings of the blobs dataset with 12 blobs having 10K points 1000-dimensional points in total are given in Fig. 1. We can see that despite the correct parameter cuML tSNE produces strangely erroneous output. We can also observe from the pictures that cuML UMAP does not produce very smooth round blobs, while the original UMAP does. Also, DiRe-JAX produces a visually pleasing embedding with round blobs having smooth boundaries.

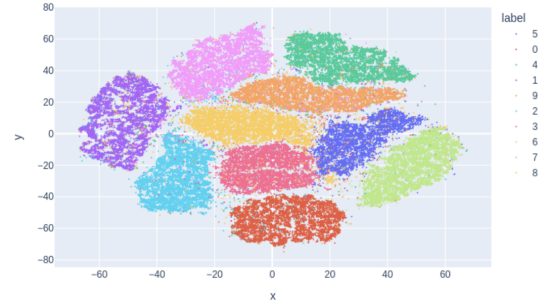
The key metrics are given in Appendix 7, from which we can see that all embeddings, except tSNE, have almost perfect scores on each global metric. The tSNE embedding (from cuML) performs badly on most global metrics, as it did come out consistently wrong in our numerical experiments.

The local metrics are different: DiRe-JAX has the largest local stress out of all, while the lowest belongs to the original UMAP.

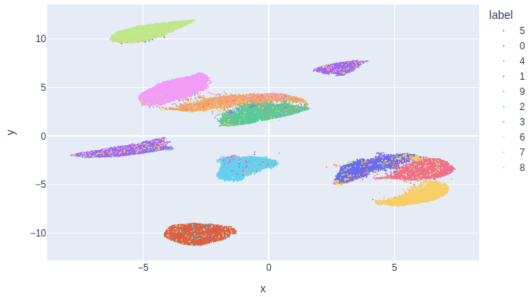
The average neighborhood preservation score is very low for all four methods (less than 1%). This can be easily explained with the following picture: a point cloud in 2D with the points $X_k = \{x_n = (1/n, n) : n = 1, \dots, k\}$ has x_k as the largest distance from the origin, yet x_k projects to the closest point to 0 on the horizontal axis.



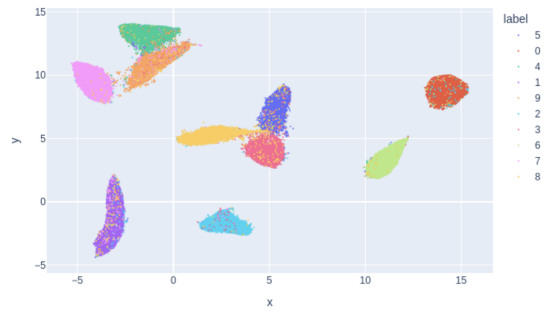
(a) DiRe – JAX



(b) cuML tSNE



(c) cuML UMAP



(d) Original UMAP

Figure 2: "MNIST Digits" dataset embeddings

5.2 Dataset: MNIST Digits

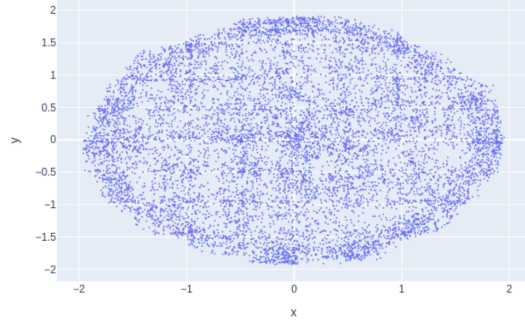
Apparently the best embedding (at least in the sense of global structure preservation) for MNIST Digits is given by tSNE. cuML UMAP rips up some clusters containing the same digit embeddings. Both cuML UMAP and the original UMAP pack a few foreign digits inside each cluster. All the while DiRe-JAX shows a smoother transition between clusters of digits that can be written similarly.

The key metrics are given in Appendix 8, which shows that tSNE performs much better on persistence homology global measures. The difference is drastic in dimension 0, so that it appears tSNE is winning in this case in plotting the actual topological layout of MNIST digits.

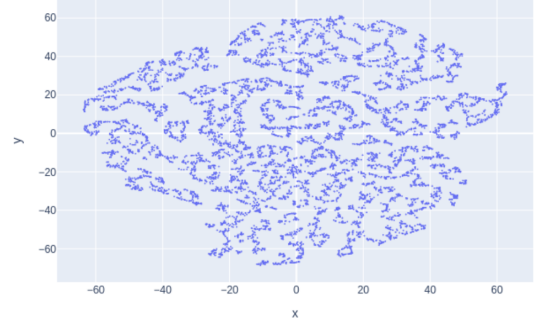
Even though all methods seem to reasonably preserve context, DiRe-JAX notably preserve both the SVM and kNN context well, while cuML tSNE preserves the SVM context almost ideally.

Also, tSNE has a much higher average neighborhood preservation score, none of the methods has a very high one (all under 6%).

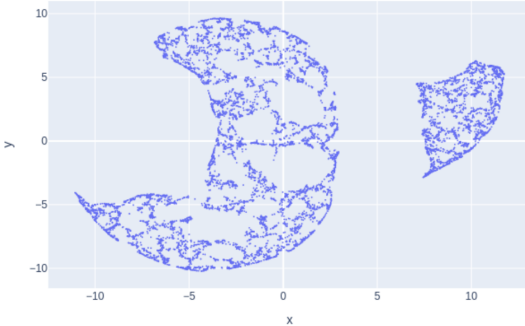
The embedding stress of all methods, especially DiRe-JAX and tSNE, is relatively high: ≈ 3.8 for DiRe-JAX, ≈ 3.5 for tSNE, and ≈ 2.7 for cuML UMAP and the original UMAP.



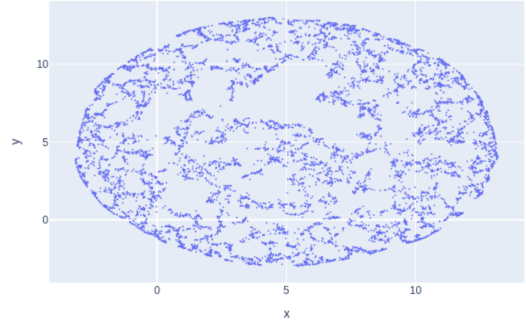
(a) DiRe - JAX



(b) cuML tSNE



(c) cuML UMAP



(d) Original UMAP

Figure 3: "Disk Uniform" dataset embeddings

5.3 Dataset: Disk Uniform

Here we embed a 2D disk with points uniformly distributed in it. In the earnest, one would expect that uniformity is preserved, especially by UMAP, as in **Uniform** Manifold Approximation and Projection, and as it purports to make the distribution more uniform by rescaling [MHM18]. However, except for DiRe-JAX none of the methods visibly produces anything close to uniformity. Even more prominently, cuML UMAP fails to produce a connected embedding for one of the major clusters.

A very similar picture arises when we try to embed a sphere and an ellipsoid in 3D with uniformly distributed points. The ellipsoid case is most egregious, as dilation in the principal axes leads to even more distortion of uniformity in the embeddings (both 2D and 3D). See the GitHub repository for more datasets [KR25].

The metrics are given in Appendix 9: here DiRe-JAX takes the lead on global metrics (associated to persistence). Note that tSNE and cuML UMAP / original UMAP outperform DiRe-JAX on the local metrics, but the performance turns out to be quite poor anyway.

5.4 Dataset: Half-moons

An interesting observation here is that DiRe-JAX preserves the context as compared to the original UMAP. Indeed, the half-moons in the input dataset cannot be separated by a straight line. In the original UMAP embedding they can, which means that some of the context is actually lost. The context preservation score supports this conclusion (see Appendix 10).

The DiRe-JAX embedding very convincingly preserves the global mutual position. The other embeddings, tSNE and cuML UMAP, turn out to be totally disjoint and loosing all of the global structure despite comparable local performance.

5.5 Dataset: Levine 13

The general situation with this dataset is close to that with MNIST Digits, however it serves as a different benchmark that represents a more “serious” dataset then the “toy” one from MNIST.

More precisely, cuML tSNE outperforms all other methods on most persistence metrics for dimension 0, except the bottleneck distance.

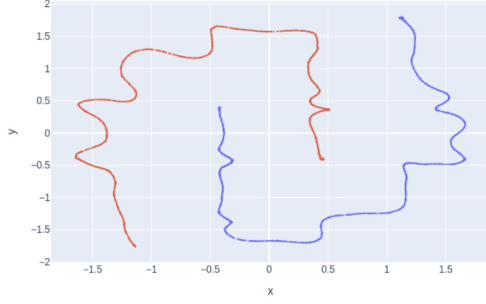
Both UMAP and cuML UMAP are clearly outperformed by DiRe – JAX on the level of global structure. Also, DiRe – JAX seems to better perform on the context preservation measures.

More benchmarking information is available in Appendix 11.

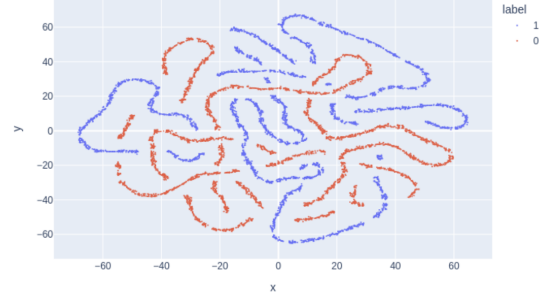
5.6 Dataset: Levine 32

In this dataset, the only embedding that shows two different strains of cells coming from two different patients is the DiRe-JAX embedding.

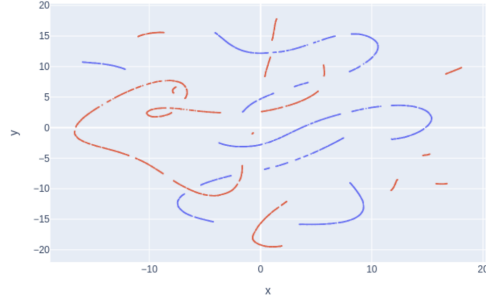
The visual inspection of other embeddings should quickly bring the reader to a conclusion that the global structure is lost. Here, DiRe – JAX is the only method whose application does not



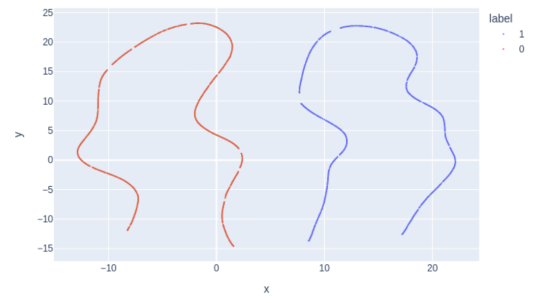
(a) DiRe – JAX



(b) cuML tSNE



(c) cuML UMAP



(d) Original UMAP

Figure 4: "Two Half-Moons" dataset embeddings

destroy, at the very least, the global structure of the dataset that clearly bears two different strata.

Interestingly (and confusingly enough) some persistence metrics for the original UMAP seems to outperform DiRe – JAX even though it is impossible to tell apart the two strata each sampled from a different individual. Also, some persistence metrics for cuML UMAP and tSNE seem comparable to those of DiRe – JAX despite the clear differences between the visuals.

More benchmarking data is available in Appendix 12.

6 Future work

The following points are important for future work and the subsequent releases of DiRe–JAX.

1. Adding more quantitative measures for analysis and optimization, as well as hyperparameter optimization functionality (suitable for smaller point clouds).
2. Improving the performance of the persistent homology computation. Since we only use dimension 0 (connected components) and dimension 1 (abelianized homotopy type) homology, we should be able to compute them more efficiently than the general algorithms available.
3. Using other distances than the L_2 distance for the kNN –graph computation and in the embedding space, as well.

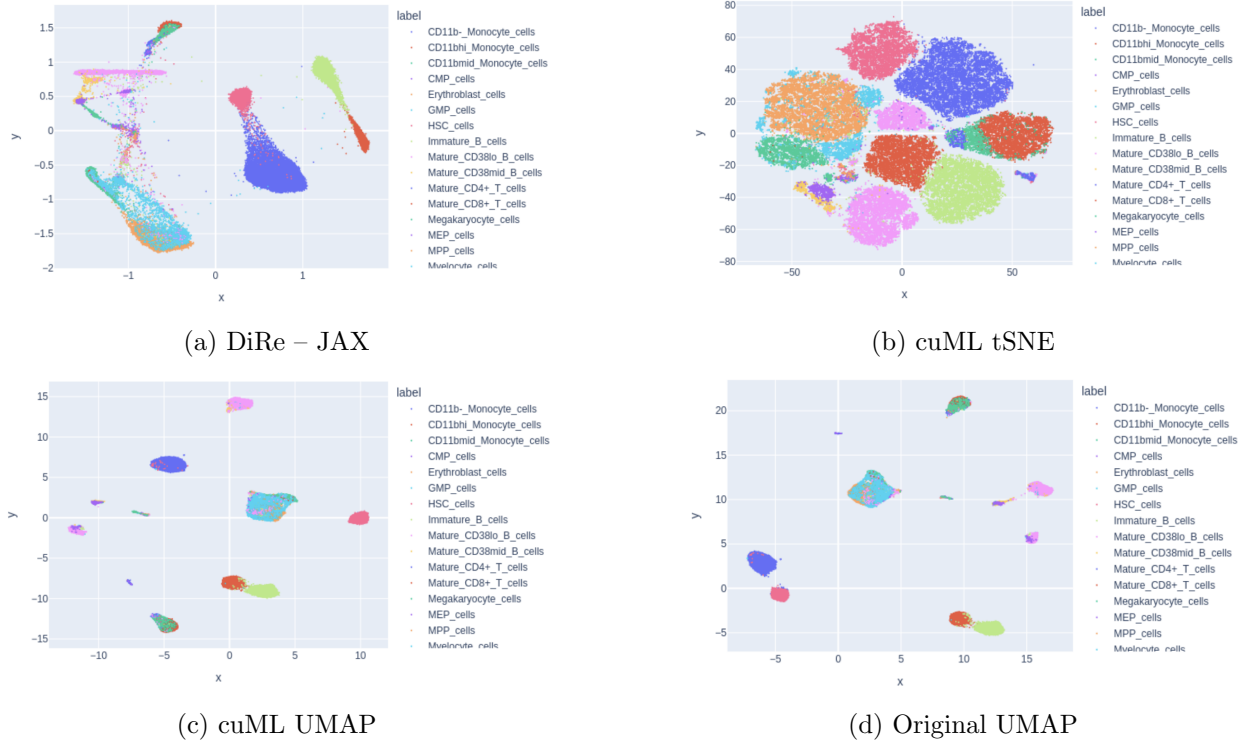


Figure 5: "Levine 13" dataset embeddings

Acknowledgments

This work is supported by the Google Cloud Research Award number GCP19980904.

References

- [JL84] William B. Johnson and Joram Lindenstrauss. "Extensions of Lipschitz mappings into a Hilbert space". In: *Contemporary Mathematics* 26 (1984), pp. 189–206. DOI: 10.1090/conm/026. URL: <https://doi.org/10.1090/conm/026>.
- [ZC05] Afra Zomorodian and Gunnar Carlsson. "Computing Persistent Homology". In: *Discrete & Computational Geometry* 33.2 (2005), pp. 249–274. DOI: 10.1007/s00454-004-1146-y. URL: <https://doi.org/10.1007/s00454-004-1146-y>.
- [Zom05] Afra J. Zomorodian. *Topology for Computing*. New York, NY, USA: Cambridge University Press, 2005. DOI: 10.1017/CB09780511546945. URL: <https://doi.org/10.1017/CB09780511546945>.
- [SC07] Stan Salvador and Philip Chan. "Toward Accurate Dynamic Time Warping in Linear Time and Space". In: *Intelligent Data Analysis* 11.5 (2007), pp. 561–580. URL: <https://doi.org/10.3233/IDA-2007-11508>.

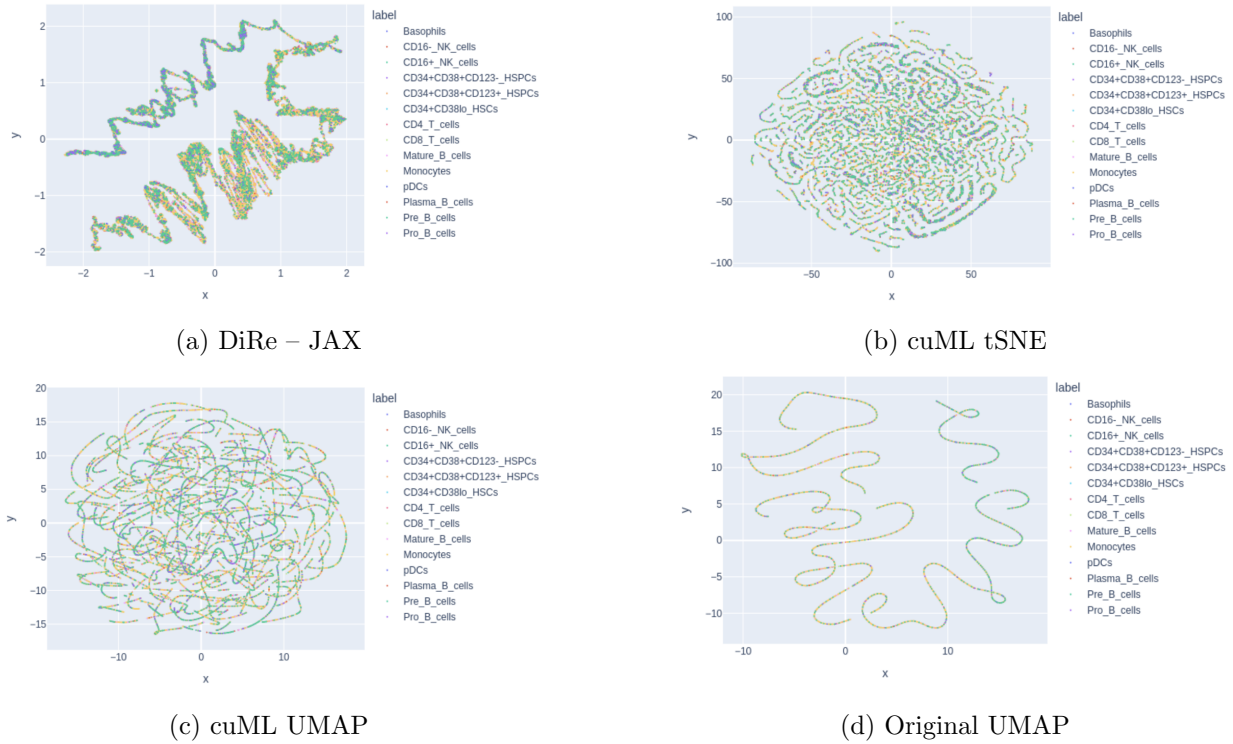
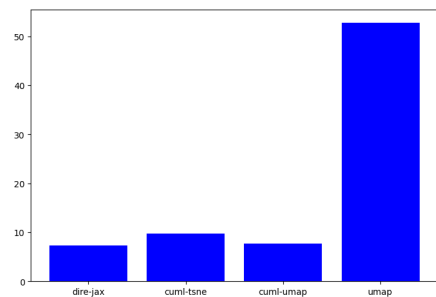


Figure 6: "Levine 32" dataset embeddings

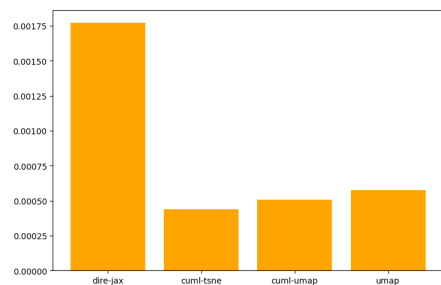
- [Mar09] Pierre-François Marteau. "Time Warp Edit Distance with Stiffness Adjustment for Time Series Matching". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 31.2 (2009), pp. 306–318. DOI: 10.1109/TPAMI.2008.76. URL: <https://doi.org/10.1109/TPAMI.2008.76>.
- [CSO14] Frédéric Chazal, Vin de Silva, and Steve Oudot. "Persistence stability for geometric complexes". In: *Geometriae Dedicata* 173.1 (2014), pp. 193–214. DOI: 10.1007/s10711-013-9937-z. URL: <https://doi.org/10.1007/s10711-013-9937-z>.
- [GGB16] Chad Giusti, Robert Ghrist, and Danielle S. Bassett. "Two's company, three (or more) is a simplex". In: *Journal of Computational Neuroscience* 41.1 (2016), pp. 1–14. DOI: 10.1007/s10827-016-0608-6. URL: <https://doi.org/10.1007/s10827-016-0608-6>.
- [MHM18] Leland McInnes, John Healy, and James Melville. "UMAP: Uniform Manifold Approximation and Projection". In: *Journal of Open Source Software* 3.29 (2018), p. 861. DOI: 10.21105/joss.00861. URL: <https://doi.org/10.21105/joss.00861>.
- [KB19] Dmitry Kobak and Philipp Berens. "The art of using t-SNE for single-cell transcriptomics". In: *Nature Communications* 10.1 (2019), p. 5416. DOI: 10.1038/s41467-019-13056-x. URL: <https://doi.org/10.1038/s41467-019-13056-x>.

- [Fla+21] Rémi Flamary et al. “POT: Python Optimal Transport”. In: *Journal of Machine Learning Research* 22.78 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-451.html>.
- [Zum21] Marcus Voßand Jonathan Zumer. *PyTWED: Python wrapper for Time Warped Edit Distance*. <https://github.com/jzumer/pytwed>. 2021.
- [CP23] Tara Chari and Lior Pachter. “The specious art of single-cell genomics”. In: *PLoS Computational Biology* 19.8 (2023), e1011288. DOI: 10.1371/journal.pcbi.1011288. URL: <https://doi.org/10.1371/journal.pcbi.1011288>.
- [Dev23] FastDTW Developers. *FastDTW: A Python Implementation of Fast Dynamic Time Warping*. <https://github.com/slaypni/fastdtw>. 2023.
- [Dou+24] Matthijs Douze et al. *The FAISS library*. 2024. arXiv: 2401.08281 [cs.LG].
- [Per24] Persim Developers. *Persim: Distances and representations of persistence diagrams*. <https://github.com/scikit-tda/persim>. 2024.
- [Dev25a] Rapids.ai cuML Developers. *cuML - GPU Machine Learning Algorithms*. <https://github.com/rapidsai/cuml/>. 2025.
- [Dev25b] UMAP Developers. *Uniform Manifold Approximation and Projection (UMAP)*. <https://github.com/lmcinnes/umap/>. 2025.
- [KR25] Alexander Kolpakov and Igor Rivin. *DiRe: Dimensionality Reducer*. <https://github.com/sashakolpakov/dire-jax>. 2025.
- [POT25] POT Developers. *POT: Python Optimal Transport*. <https://github.com/PythonOT/POT>. 2025.

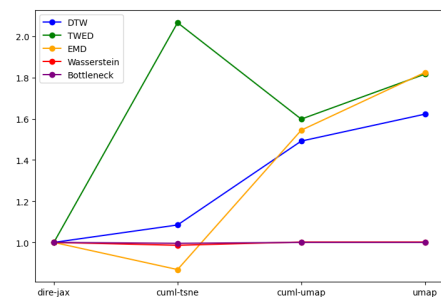
7 Blobs: Dataset Metrics



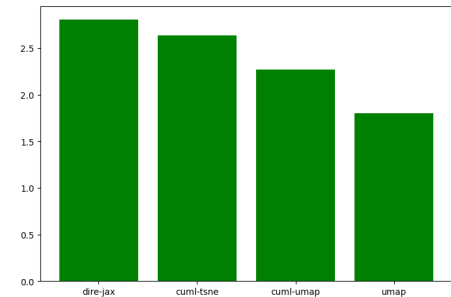
(a) Runtime (seconds)



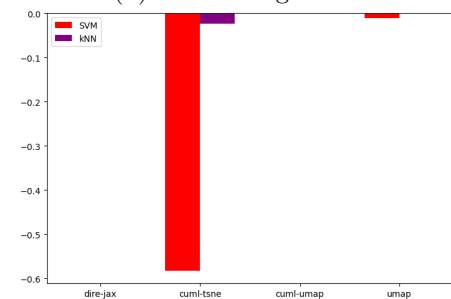
(c) Neighborhood preservation



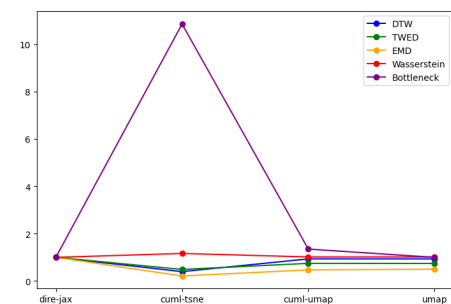
(e) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



(b) Embedding stress



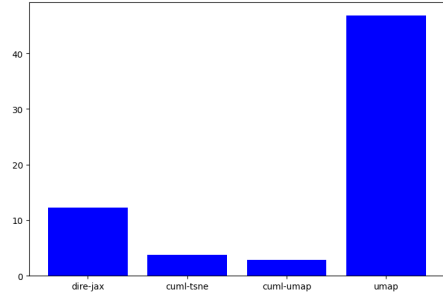
(d) Context preservation



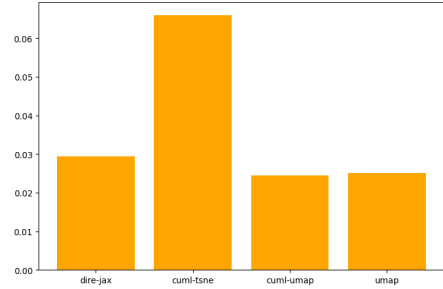
(f) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 7: "Blobs" dataset metrics comparison

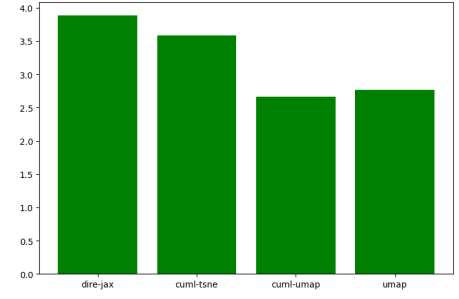
8 MNIST Digits: Dataset Metrics



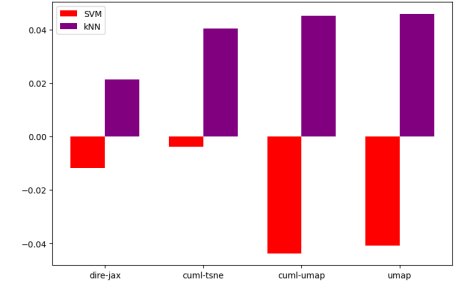
(a) Runtime (seconds)



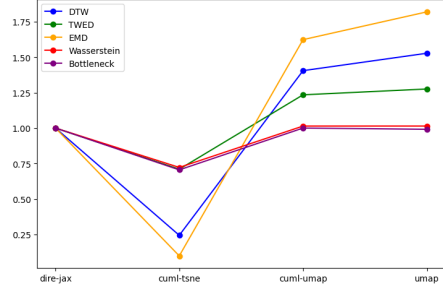
(c) Neighborhood preservation



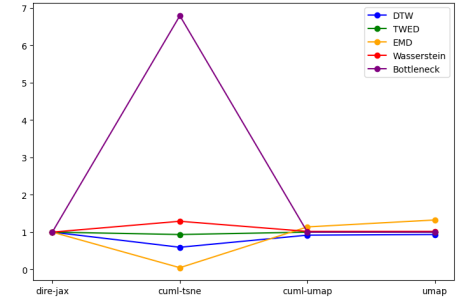
(b) Embedding stress



(d) Context preservation



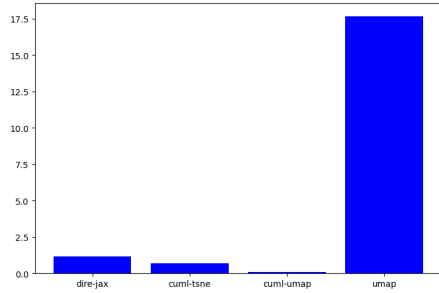
(e) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



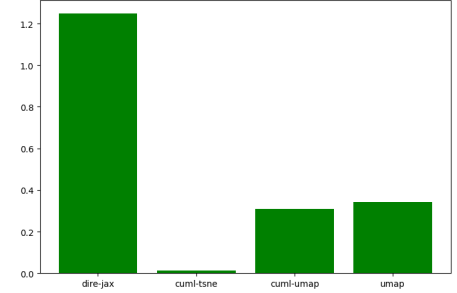
(f) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 8: "MNIST Digits" dataset metrics comparison

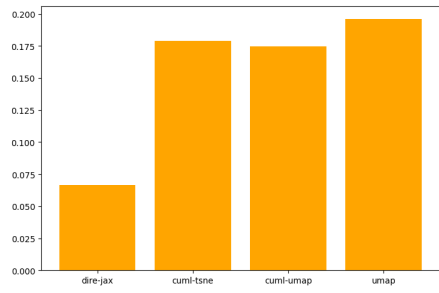
9 Disk Uniform: Dataset Metrics



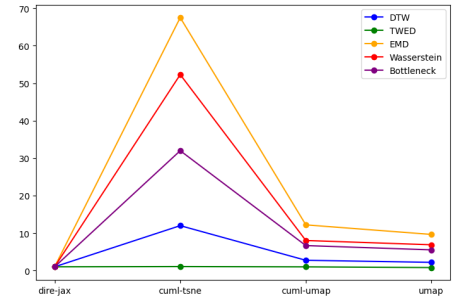
(a) Runtime (seconds)



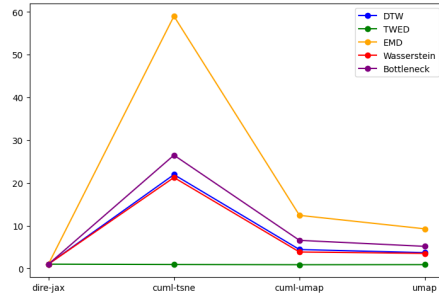
(b) Embedding stress



(c) Neighborhood preservation



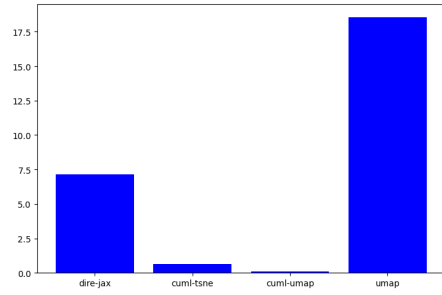
(d) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



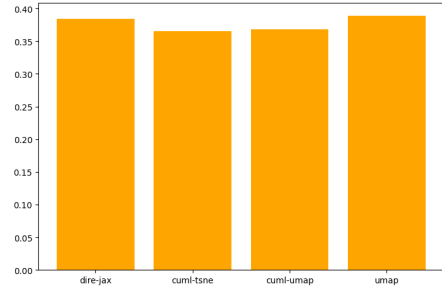
(e) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 9: "Disk Uniform" dataset metrics comparison

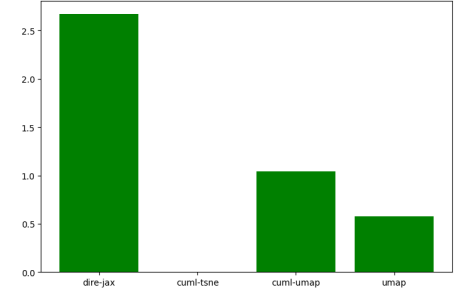
10 Half-moons: Dataset Metrics



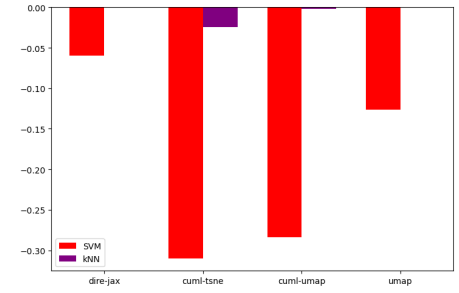
(a) Runtime (seconds)



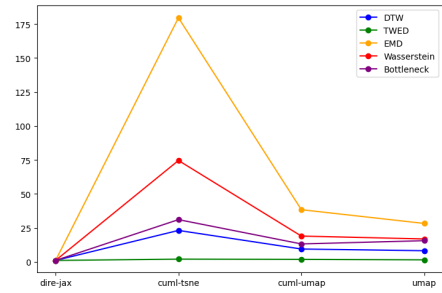
(c) Neighborhood preservation



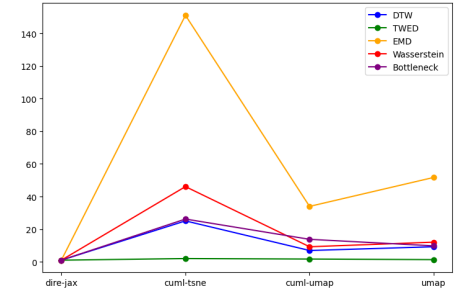
(b) Embedding stress



(d) Context preservation



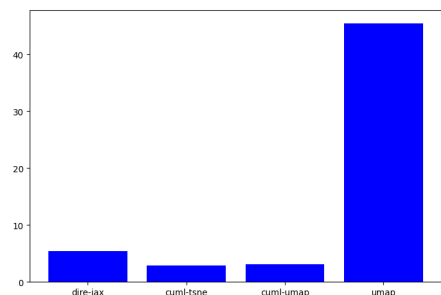
(e) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



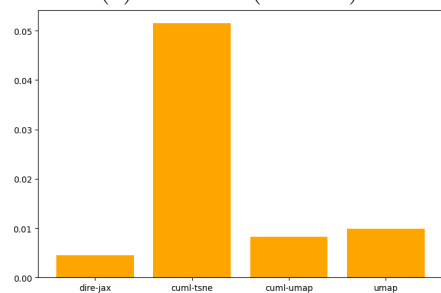
(f) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 10: "Half-moons" dataset metrics comparison

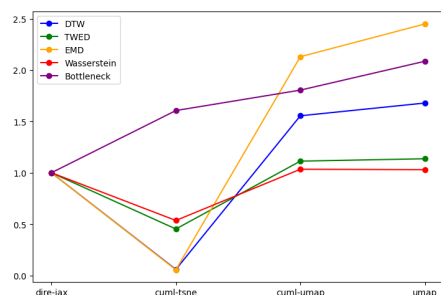
11 Levine 13: Dataset Metrics



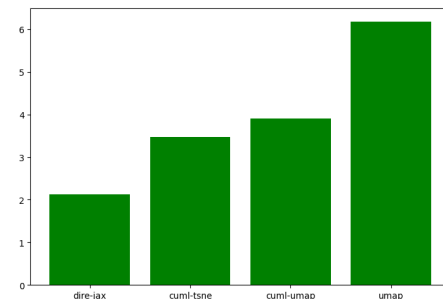
(a) Runtime (seconds)



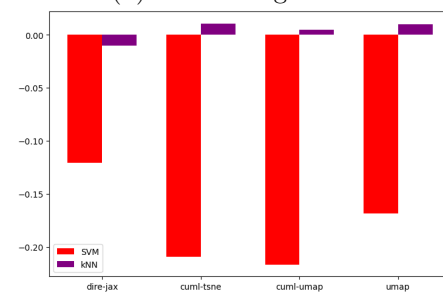
(c) Neighborhood preservation



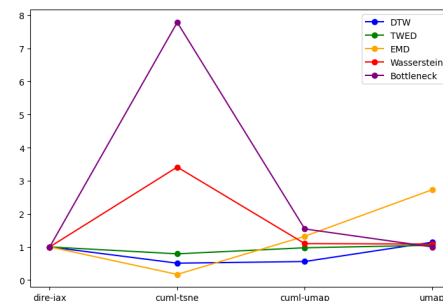
(e) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



(b) Embedding stress



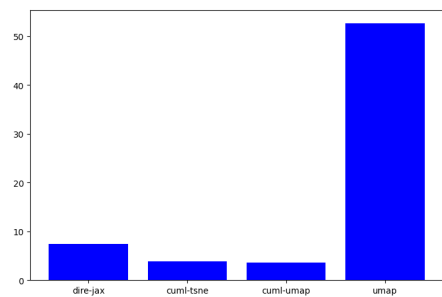
(d) Context preservation



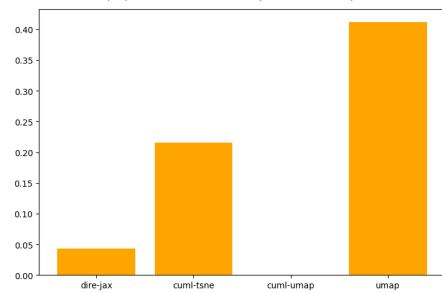
(f) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 11: "Levine 13" dataset metrics comparison

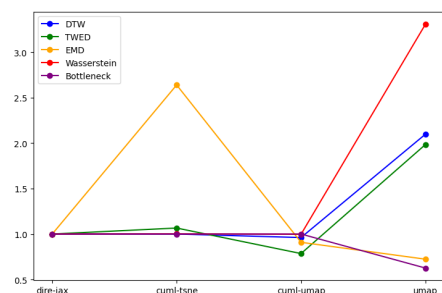
12 Levine 32: Dataset Metrics



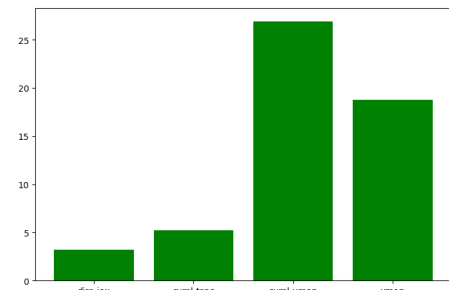
(a) Runtime (seconds)



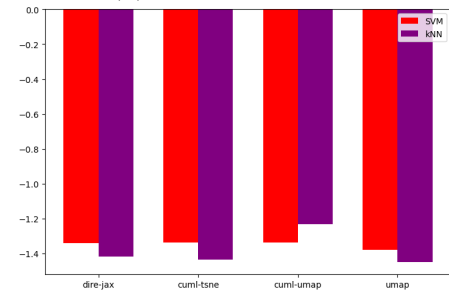
(c) Neighborhood preservation



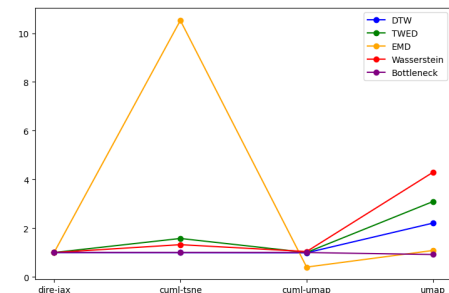
(e) Persistence homology preservation: dimension 0 (relative to DiRe-JAX baseline)



(b) Embedding stress



(d) Context preservation



(f) Persistence homology preservation: dimension 1 (relative to DiRe-JAX baseline)

Figure 12: "Levine 32" dataset metrics comparison